Link to the source code: GitHub Repository

Authors

- Wojciech Bogacz 156034
- Krzysztof Skrobała 156039

Problem Description

The problem considered in this project involves a set of nodes, each represented by three integer values: two coordinates (x_i)(x_i, y)(x_i) that define the node's position in a two-dimensional plane, and a cost value associated with the node. The objective is to select exactly 50% of all available nodes (if the total number of nodes is odd, the number of selected nodes is rounded up) and construct a Hamiltonian cycle — that is, a closed path visiting each selected node exactly once and returning to the starting node. The goal is to minimize the sum of *two components*: 1. The total length of the constructed cycle, and 2. The total cost of all selected nodes The distances between nodes are calculated using the Euclidean metric, rounded to the nearest integer.

Implemented Algorithms

```
FUNCTION LNS_Algorithm(instance, initialTour, isWithSearch)
    currentTour = initialTour

    IF isWithSearch IS True THEN
        currentTour = RunLocalSearch(currentTour, "steepest", "edges")

    bestTour = currentTour

    WHILE currentTime < averageMSLoTime DO
        destroyTime = instance.N / 2
        partialTour = Destroy_Subpath(currentTour, destroyTime)
        currentTour = Repair_Regret(partialTour, instance)

        IF isWithSearch IS true THEN
            currentTour = RunLocalSearch(currentTour, "steepest",
"edges")

        IF Cost(currentTour) < Cost(bestTour) THEN
            bestTour = currentTour

    RETURN bestTour
END FUNCTION



FUNCTION Destroy_Subpath(tour, targetSize)
    removedNodes = empty set
    WHILE size(removedNodes) < targetSize
        startIdx = Random(0, size(tour) - 1)
        pathLen = 3 + Random(0, 3)
        FOR i FROM 0 TO pathLen - 1
            idx = (startIdx + i) MOD size(tour)
            Add tour[idx] to removedNodes
            IF size(removedNodes) >= targetSize THEN BREAK

    partialTour = empty list
    FOR EACH node IN tour
        IF node NOT IN removedNodes THEN
            Append node TO partialTour

    RETURN partialTour
END FUNCTION


FUNCTION Repair_Regret(partialTour, instance)
    WHILE size(partialTour) < instance.N
        candidates = empty list
        FOR EACH node NOT IN partialTour
            bestInc, bestPos = CalculateBestInsertion(node,
partialTour)
            secondInc = CalculateSecondBestInsertion(node,
partialTour)

            bestTotal = bestInc + node.Cost
            secondTotal = secondInc + node.Cost
            regret = secondTotal - bestTotal
            score = regret - bestTotal

            Add (node, score, bestPos) TO candidates

        bestCandidate = candidate WITH MAX score
        Insert bestCandidate.node INTO partialTour AT
bestCandidate.bestPos

    RETURN partialTour
END FUNCTION
```

Results

Comparison of previous methods results

Instance A

Method	Best	Worst	Average
random	235308	292123	264677.17
nn_end	89198	123123	104612.785
nn_anywhere	71488	74410	72635.72
greedy cycle	72639	72639	72639.0
2-Regret Insertion	105852	123428	115474.93
Weighted Sum (a=1.00, β=1.00)	71108	73438	72130.85
greedy_intra-edges_start-greedy	70663	72648	71594.055
greedy_intra-edges_start-random	71564	76328	73528.81
greedy_intra-nodes_start-greedy	70687	73027	71739.395
greedy_intra-nodes_start-random	79677	99035	86909.13
steepest_intra-edges_start-greedy	70510	72614	71463.08
steepest_intra-edges_start-random	71246	78153	73699.995
steepest_intra-nodes_start-greedy	70626	73004	71615.93
steepest_intra-nodes_start-random	81322	95342	87939.335
steepest_intra-edges_start-random	71246	78153	73699.995
steepest_intra-edges_start-random with Cand. mech.	73876	85487	77866.91
steepest_lm_intra-edges_start-random	71265	78247	73856.275
ILS_intra-edges_start-random	70901	71862	71303.60
steepest_multi_start_intra-edges_start-random	71149	72794	71299.05

New Methods Results

Method	best	worst	average
LNS_no_search_intra-edges_start-random	69732	71868	70604.2
LNS_with_search_intra-edges_start-random	69397	71604	70453.9

Number of main loop iterations

Method	best	worst	average
LNS_no_search_intra-edges_start-random	948	1073	1013.35
LNS_with_search_intra-edges_start-random	823	935	899.10

Instance B

Method	Best	Worst	Average
random	193234	234798	214279.235
nn_end	62606	77453	69764.43
nn_anywhere	49001	57324	51400.905
greedy cycle	50243	50243	50243.0
2-Regret Insertion	66505	77872	72454.77
Weighted Sum (a=1.00, β=1.00)	47144	55700	50918.82
greedy_intra-edges_start-greedy	46178	55471	50172.67
greedy_intra-edges_start-random	45537	51687	48154.865
greedy_intra-nodes_start-greedy	46373	55385	50347.525
greedy_intra-nodes_start-random	53417	71474	61453.295
steepest_intra-edges_start-greedy	45867	54814	49907.285
steepest_intra-edges_start-random	45903	51416	48195.845
steepest_intra-nodes_start-greedy	46371	55385	50201.47
steepest_intra-nodes_start-random	55686	71546	62955.885
steepest_intra-edges_start-random	45903	51416	48195.845
steepest_intra-edges_start-random with Cand. Mech.	45798	52670	48474.64
steepest_lm_intra-edges_start-random	45941	51587	48214.715
ILS_intra-edges_start-random	45320	46753	45463.3
steepest_multi_start_intra-edges_start-random	45405	46222	45767.4

New Methods Results

Method	best	worst	average
LNS_no_search_intra-edges_start-random	44415	46341	45336.95
LNS_with_search_intra-edges_start-random	44012	45376	44511.30

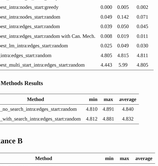
Number of main loop iterations

Method	best	worst	average
LNS_no_search_intra-edges_start-random	921	1028	984.35
LNS_with_search_intra-edges_start-random	835	1005	892.30

Paths visualization

Instance A

LNS without local search variant



LNS with local search variant

Instance B

LNS without local search variant

LNS with local search variant

Time

Instance A

Method	min	max	average
greedy_intra-edges_start-greedy	0.00	0.006	0.003
greedy_intra-edges_start-random	0.062	0.099	0.080
greedy_intra-nodes_start-greedy	0.000	0.005	0.002
greedy_intra-nodes_start-random	0.066	0.118	0.085
steepest_intra-edges_start-greedy	0.001	0.005	0.002
steepest_intra-nodes_start-greedy	0.000	0.005	0.002
steepest_intra-edges_start-random	0.049	0.142	0.071
steepest_intra-nodes_start-random	0.039	0.050	0.045
steepest_intra-edges_start-random with Can. Mech.	0.008	0.019	0.011
steepest_lm_intra-edges_start-random	0.025	0.049	0.030
ILS_intra-edges_start-random	4.805	4.815	4.811
steepest_multi_start_intra-edges_start-random	4.443	5.99	4.805

New Methods Results

Method	min	max	average
LNS_no_search_intra-edges_start-random	4.810	4.891	4.840
LNS_with_search_intra-edges_start-random	4.812	4.881	4.832

Instance B

Method	min	max	average
greedy_intra-edges_start-greedy	0.0	0.01	0.004
greedy_intra-edges_start-random	0.064	0.102	0.081
greedy_intra-nodes_start-greedy	0.0	0.007	0.003
greedy_intra-nodes_start-random	0.058	0.107	0.082
steepest_intra-edges_start-greedy	0.001	0.01	0.004
steepest_intra-nodes_start-greedy	0.001	0.007	0.004
steepest_intra-nodes_start-random	0.048	0.086	0.063
steepest_intra-edges_start-random	0.038	0.054	0.045
steepest_intra-edges_start-random with CAn. Mech.	0.009	0.05	0.012
steepest_lm_intra-edges_start-random	0.020	0.053	0.035
ILS_intra-edges_start-random	4.499	4.515	4.503
steepest_multi_start_intra-edges_start-random	5.377	5.477	4.458

New Methods Results

Method	min	max	average
LNS_no_search_intra-edges_start-random	4.499	4.609	4.554
LNS_with_search_intra-edges_start-random	4.501	4.607	4.540

Conclusions

LMS performed the best among all tested methods, achieving the lowest average costs for both instances A and B. Its strategy of exploring multiple starting points allowed it to escape local minima effectively, leading to superior solutions. Moreover, its computational time was reasonable given the quality of solutions obtained.